Interactions Application ↔ Serveur API — Documentation technique

Ce document recense l'ensemble des échanges entre l'application Flutter Nestvia et le serveur API REST (`https://api.leofranz.fr/nestvia`).

> **Convention** : tous les endpoints sont préfixés par `/nestvia`. Tous requièrent un **JWT Bearer** sauf `/auth/login` et `/tentatives`.

Table des matières

1. Démarrage de l'application (Splash Screen)
2. Page d'accueil non-connectée (Landing)
3. Authentification (Login)
4. Page d'accueil connectée (Home)
5. Recherche avancée avec filtres
6. Carte interactive (Map)
7. Réservation d'un bien (Booking)
8. Favoris
9. Mes réservations
10. Notifications
11. Dépôt d'avis
12. Profil utilisateur
13. Déconnexion
14. Synthèse des endpoints

---

1 — Démarrage de l'application (Splash Screen)

Écran : `SplashScreen`
Fichier : `lib/screens/splash_screen.dart`

Flux détaillé

1. L'utilisateur lance l'application.
2. Le `SplashScreen` s'affiche avec le logo Nestvia et un indicateur de chargement.
3. L'application vérifie si un token JWT est déjà stocké dans le stockage sécurisé (`FlutterSecureStorage`).
4. **Si un token existe** :
   - Le token est réinjecté dans les headers HTTP du client Dio via `ApiService().setAuthToken(token)`.
   - L'utilisateur est redirigé automatiquement vers la **page d'accueil connectée** (`MainNavScreen`).
5. **Si aucun token n'existe** :
   - L'utilisateur est redirigé vers la **page d'accueil non-connectée** (`LandingScreen`).

> **Aucun appel API n'est effectué** à cette étape. La vérification est purement locale (lecture du stockage sécurisé).

```
┌──────────────────┐
│  SplashScreen    │
└──────────────────┘
        │
        │  Lecture FlutterSecureStorage
        ├── Token trouvé ──────> MainNavScreen (accueil connecté)
        └── Pas de token ──────> LandingScreen (accueil non-connecté)
```

---

2 — Page d'accueil non-connectée (Landing)

Écran : `LandingScreen`
Fichier : `lib/screens/landing_screen.dart`

Flux détaillé

1. L'utilisateur voit le logo Nestvia, un bouton **« Se connecter »** et un bouton **« S'inscrire »**.
2. **Bouton « Se connecter »** :
   - Navigation interne vers l'écran `AuthScreen` (formulaire de login).
3. **Bouton « S'inscrire »** :
   - Ouverture du navigateur externe vers l'URL `https://nestvia.leofranz.fr/connexion_inscription.php`.
   - L'inscription se fait entièrement côté site web. L'application n'intervient pas dans ce processus.

> **Aucun appel API n'est effectué** sur cette page. Les actions sont soit une navigation interne, soit l'ouverture d'une URL externe.

---

3 — Authentification (Login)

Écran : `AuthScreen`
Fichier : `lib/screens/auth_screen.dart`
Service : `AuthService` (`lib/services/auth_service.dart`)

Flux détaillé

1. L'utilisateur saisit son adresse email et son mot de passe dans le formulaire de connexion.
2. **Validation côté client** (avant tout appel API) :
   - L'email doit respecter un format valide (regex : `^[\w\-.]+@([\w\-]+\.)+[\w\-]{2,4}$`).
   - Le mot de passe ne doit pas être vide.
   - Si la validation échoue, les erreurs sont affichées sous les champs. Aucun appel API n'est envoyé.
3. L'utilisateur clique sur **« Se connecter »**.
4. L'application transforme les données en JSON et effectue un appel `POST` vers l'endpoint API `/nestvia/auth/login`.

Requête API

```
POST /nestvia/auth/login
Content-Type: application/json
```

```
{
  "email": "utilisateur@exemple.com",
  "password": "monMotDePasse"
}
```

5. L'API récupère les données et compare les informations en base de données (MariaDB) avec celles reçues.
6. **Si les identifiants sont corrects** (code HTTP `200`) :
   - L'API génère un **token JWT** signé avec la clé secrète (`JWT_SECRET`), valide pendant 24 heures (`JWT_EXPIRES_IN`).
   - L'API renvoie le token dans la réponse JSON :

     ```
     { "token": "eyJhbGciOiJIUzI1NiJ9.eyJpZCI6NDJ9.SflKxw..." }
     ```

   - L'application extrait le token de la réponse (`response.data['token']`).
   - Le token est stocké de manière sécurisée dans `FlutterSecureStorage` sous la clé `auth_token` (Android Keystore / iOS Keychain).
   - Le token est injecté dans le client HTTP Dio : `Authorization: Bearer <token>`.
   - L'utilisateur est redirigé vers la **page d'accueil connectée** (`MainNavScreen`).
7. **Si les identifiants sont incorrects** (code HTTP `401` ou `403`) :
   - L'API renvoie un code d'erreur.
   - L'application affiche le message : *« Email ou mot de passe incorrect. Compte inexistant ou verrouillé. »*
8. **En cas d'erreur réseau ou serveur** :
   - L'erreur est traduite en message lisible via `ApiService.handleError()` (timeout, serveur indisponible, etc.).
   - Le message est affiché à l'utilisateur dans le formulaire.

> **Chaque appel API autre que `/auth/login` et `/tentatives` nécessite le Bearer**, renvoyé par l'application à chaque requête dans le header `Authorization`.

Liens additionnels sur l'écran de login

- **« Mot de passe oublié ? »** : ouvre le navigateur externe vers `https://nestvia.leofranz.fr/connexion_inscription.php`. Aucun appel API.
- **« Pas encore de compte ? S'inscrire »** : ouvre le navigateur externe vers `https://nestvia.leofranz.fr/connexion_inscription.php`. Aucun appel API.

```
┌──────────────┐                                  ┌──────────────┐
│  AuthScreen  │  POST /nestvia/auth/login        │   Serveur    │
│  (Flutter)   │  { email, password }             │   API REST   │
│              │ ──────────────────────────────>  │              │
│              │                                  │  Vérifie en  │
│              │                                  │  base BDD    │
│              │  200 { "token": "eyJ..." }       │              │
│              │ <──────────────────────────────  │              │
```

```
  |               |                                  └────────────────┘
  |  Stocke JWT   |
  |  dans Secure  |
  |  Storage      |
  |               |
  |  Injecte le   |
  |  Bearer dans  |
  |  Dio headers  |
  |               |──────> MainNavScreen
  └───────────────┘
```

---

4 — Page d'accueil connectée (Home)

Écran : `HomeScreen`
Fichier : `lib/screens/home_screen.dart`
Services : `PropertyService`, `ProfileService`, `NotificationService`, `FavoriteService`, `LocationService`

Flux détaillé

Au chargement de la page d'accueil, **6 opérations sont lancées en parallèle** :

4.1 — Chargement des biens « En vedette »

1. L'application effectue un appel `GET /nestvia/biens` (sans filtre).
2. L'API renvoie la liste de tous les biens avec un code `200`.
3. Pour **chaque bien** de la liste, l'application effectue **deux appels supplémentaires en parallèle** pour enrichir les données :
   - `GET /nestvia/biens/:id/photos` → récupération de la première photo du bien.
   - `GET /nestvia/biens/:id/tarifs` → récupération du tarif hebdomadaire (semaine courante ou premier disponible).
4. Les biens enrichis (avec photo + tarif) sont affichés dans le carrousel horizontal « En vedette ».

```
GET /nestvia/biens
  → 200 : [ { id_bien: 1, nom_bien: "...", ... }, ... ]

Pour chaque bien :
  GET /nestvia/biens/1/photos  → 200 : [ { lien_photo: "https://..." }, ... ]
  GET /nestvia/biens/1/tarifs  → 200 : [ { tarif: 250, annee_tarif: 2026, ...
}, ... ]
```

4.2 — Chargement des biens « Près de vous »

1. L'application demande la position GPS de l'utilisateur via le plugin `Geolocator`.
   - Si la permission est refusée ou le GPS désactivé, la section affiche *« Position indisponible »*.
2. Si la position est obtenue, l'application effectue un appel `GET /nestvia/biens` avec les coordonnées GPS en paramètres.

```
GET /nestvia/biens?lat=43.6047&lng=1.4442
  → 200 : [ { id_bien: 5, nom_bien: "...", distance_km: 2.3, ... }, ... ]
```

3. Les biens reçus sont enrichis (photo + tarif) comme pour les biens « En vedette ».
4. Les résultats sont affichés dans la section « Près de vous ».

4.3 — Chargement des catégories de biens

1. L'application effectue un appel `GET /nestvia/types-bien`.
2. L'API renvoie la liste des types de biens.

```
GET /nestvia/types-bien
  → 200 : [ { id_typebien: 1, des_typebien: "Appartement" }, { id_typebien:
2, des_typebien: "Maison" }, ... ]
```

3. Les types sont ajoutés comme onglets de filtrage (« Tous », « Appartement », « Maison », « Villa »…).
4. Quand l'utilisateur sélectionne un onglet, le filtrage s'applique **localement** sur les biens déjà chargés (pas de nouvel appel API).

4.4 — Chargement des informations du profil (header)

1. L'application effectue un appel `GET /nestvia/compte`.

```
GET /nestvia/compte
Authorization: Bearer eyJ...
  → 200 : { "prenom_locataire": "Jean", "nom_locataire": "Dupont", ... }
```

2. Le prénom et les initiales sont extraits pour personnaliser le message d'accueil et l'avatar dans le header.

4.5 — Comptage des notifications non lues

1. L'application effectue un appel `GET /nestvia/notifications`.

```
GET /nestvia/notifications
Authorization: Bearer eyJ...
  → 200 : [ { id_notification: 1, is_read: 0, ... }, { id_notification: 2,
is_read: 1, ... }, ... ]
```

2. L'application compte les notifications ayant `is_read == 0`.
3. Le badge de notification dans le header affiche ce nombre.

4.6 — Chargement des favoris en cache

1. L'application effectue un appel `GET /nestvia/favoris`.

```
GET /nestvia/favoris
Authorization: Bearer eyJ...
  → 200 : [ { id_favoris: 10, id_bien: 3 }, { id_favoris: 11, id_bien: 7 },
... ]
```

2. Les IDs des biens favoris sont stockés dans un `Set<int>` en mémoire locale (cache applicatif).
3. Ce cache permet d'afficher immédiatement l'icône cœur plein/vide sur les cartes de biens sans appel API supplémentaire.

Pull-to-refresh

L'utilisateur peut tirer vers le bas pour rafraîchir la page. Cette action relance **les 4 chargements principaux** : favoris, biens en vedette, biens proches, données du header.

---

5 — Recherche avancée avec filtres

Écran : `SearchScreen`
Fichier : `lib/screens/search_screen.dart`
Services : `PropertyService`, `ApiService`

Flux détaillé

5.1 — Autocomplétion des communes

1. L'utilisateur commence à taper dans le champ « Commune » (minimum 2 caractères).
2. L'application effectue un appel `GET /nestvia/communes?search={query}`.

```
GET /nestvia/communes?search=mont
Authorization: Bearer eyJ...
  → 200 : [ { id_commune: 30438, nom_commune: "Montpellier" }, { id_commune:
12345, nom_commune: "Montauban" }, ... ]
```

3. Les résultats sont affichés en liste déroulante. L'utilisateur sélectionne une commune.
4. L'ID de la commune est mémorisé pour la requête de recherche.

5.2 — Autocomplétion des types de bien

1. L'utilisateur interagit avec le champ « Type de bien ».
2. L'application effectue un appel `GET /nestvia/types-bien`.

```
GET /nestvia/types-bien
Authorization: Bearer eyJ...
  → 200 : [ { id_typebien: 1, des_typebien: "Appartement" }, ... ]
```

3. Les résultats sont filtrés localement selon la saisie de l'utilisateur.

4. L'ID du type de bien est mémorisé pour la requête de recherche.

5.3 — Soumission de la recherche

1. L'utilisateur configure les filtres (commune, type de bien, nb de couchages, animaux, fourchette de tarif, dates de séjour) et clique sur **« Rechercher »**.
2. L'application combine les filtres actifs et effectue un appel `GET /nestvia/biens` avec les paramètres correspondants.

```
GET /nestvia/biens?
commune=30438&type_bien=2&nb_personnes=4&animaux=oui&tarif_min=100&tarif_max=
500
Authorization: Bearer eyJ...
  → 200 : [ { id_bien: 5, nom_bien: "Villa Soleil", ... }, ... ]
```

3. Les biens reçus sont enrichis (photos + tarifs) via des appels individuels `GET /biens/:id/photos` et `GET /biens/:id/tarifs` pour chaque bien.

5.4 — Filtrage par disponibilité (si dates de séjour renseignées)

4. Si l'utilisateur a renseigné des dates de séjour, l'application effectue un appel `GET /nestvia/biens/:id/disponibilite` **pour chaque bien** des résultats :

```
GET /nestvia/biens/5/disponibilite?date_debut=2026-07-05&date_fin=2026-07-19
Authorization: Bearer eyJ...
  → 200 : { "disponible": true, "reservations_conflit": [],
"blocages_conflit": [] }
```

5. Seuls les biens disponibles (`disponible: true`) sont conservés.
6. L'utilisateur est redirigé vers la **carte interactive** (`MapScreen`) avec les résultats filtrés.

---

6 — Carte interactive (Map)

Écran : `MapScreen`
Fichier : `lib/screens/map_screen.dart`
Services : `PropertyService`, `LocationService`, `FavoriteService`, `ReservationService`

Flux détaillé

6.1 — Chargement des données

1. **Deux cas possibles** :
   - **Accès depuis la recherche** : les biens sont déjà fournis en paramètre par `SearchScreen` → pas de nouvel appel API.
   - **Accès direct** (via la barre de navigation) : l'application charge les biens via `GET /nestvia/biens` et les enrichit (photos + tarifs), identique à la section 4.1.

2. La position GPS de l'utilisateur est récupérée localement via `Geolocator` (pas d'appel API).
3. Les biens ayant des coordonnées GPS sont placés comme marqueurs sur la carte OpenStreetMap (flutter_map).

6.2 — Filtrage local par prix

- L'utilisateur peut filtrer par catégorie de prix (« Tous », « < 100€ », « Luxe »).
- Ce filtrage s'applique **localement** sur les biens déjà chargés (pas de nouvel appel API).

6.3 — Popup fiche d'un bien

1. L'utilisateur clique sur un marqueur de bien sur la carte.
2. L'application effectue **deux appels en parallèle** pour charger le détail :

```
GET /nestvia/biens/5
Authorization: Bearer eyJ...
  → 200 : { id_bien: 5, nom_bien: "Villa Soleil", prestations: [...], ... }

GET /nestvia/biens/5/photos
Authorization: Bearer eyJ...
  → 200 : [ { lien_photo: "https://nestvia.leofranz.fr/photos/5_1.jpg" }, ...
]
```

3. La popup affiche : photo, nom, note, localisation, type, superficie, prestations, prix, et un bouton **« Réserver »**.

6.4 — Toggle favori depuis la carte

- L'utilisateur peut cliquer sur l'icône cœur d'un bien dans la bottom sheet.
- Voir la section 8 — Favoris pour le détail de l'interaction API.

6.5 — Navigation vers la réservation

- L'utilisateur clique sur **« Réserver »** dans la popup d'un bien.
- Navigation vers `BookingScreen` avec le bien sélectionné (et les dates si elles proviennent de la recherche).

---

7 — Réservation d'un bien (Booking)

Écran : `BookingScreen` (3 étapes)
Fichier : `lib/screens/booking_screen.dart`
Service : `ReservationService` (`lib/services/reservation_service.dart`)

Flux détaillé

7.0 — Chargement initial des données du bien

1. À l'ouverture de `BookingScreen`, l'application effectue **deux appels en parallèle** :

```
GET /nestvia/biens/5/tarifs
Authorization: Bearer eyJ...
  → 200 : [ { id_tarif: 12, tarif: 350, annee_tarif: 2026, semaine_tarif: 27
}, ... ]

GET /nestvia/biens/5/photos
Authorization: Bearer eyJ...
  → 200 : [ { lien_photo: "https://nestvia.leofranz.fr/photos/5_1.jpg" }, ...
]
```

2. Le premier tarif disponible est sélectionné par défaut. La première photo est utilisée pour le résumé du bien.

7.1 — Étape 1 : Sélection des dates et voyageurs

1. L'utilisateur sélectionne une plage de dates via le sélecteur de dates natif (`showDateRangePicker`).
2. L'utilisateur ajuste le nombre de voyageurs (compteur local, pas d'appel API).
3. Le montant est calculé **localement** : `nombre de semaines × tarif hebdomadaire + 5% frais de service`.
4. L'utilisateur clique sur **« Continuer »**.
5. L'application vérifie la disponibilité du bien pour les dates choisies :

```
GET /nestvia/biens/5/disponibilite?date_debut=2026-07-05&date_fin=2026-07-19
Authorization: Bearer eyJ...
  → 200 : { "disponible": true, "reservations_conflit": [],
"blocages_conflit": [] }
```

6. **Si le bien est disponible** (`disponible: true`) → passage à l'étape 2.
7. **Si le bien n'est pas disponible** (`disponible: false`) :
   - Le message *« Ce bien n'est pas disponible sur la période sélectionnée. Veuillez choisir d'autres dates. »* est affiché.
   - L'utilisateur reste sur l'étape 1.

7.2 — Étape 2 : Récapitulatif et calcul du montant

1. L'application affiche un récapitulatif du séjour :
   - Nom du bien, dates, nombre de nuits/semaines
   - Tarif hebdomadaire × nombre de semaines
   - Frais de service (5%)
   - **Montant total**
2. **Aucun appel API** à cette étape : les données sont déjà en mémoire.
3. L'utilisateur clique sur **« Confirmer »** → passage à l'étape 3.

7.3 — Étape 3 : Paiement et création de la réservation

1. L'utilisateur remplit le formulaire de paiement (titulaire, numéro de carte, expiration, CVV).
2. **Validation côté client** des champs du formulaire.

3. L'utilisateur clique sur **« Payer et réserver »**.
4. L'application effectue un appel `POST /nestvia/reservations` :

```
POST /nestvia/reservations
Authorization: Bearer eyJ...
Content-Type: application/json

{
  "date_debut": "2026-07-05",
  "date_fin": "2026-07-19",
  "id_bien": 5,
  "id_tarif": 12
}
```

5. **Le montant total est calculé automatiquement côté serveur** (nombre de semaines × tarif).
6. **Si la réservation réussit** (code HTTP `200` ou `201`) :
   - L'API renvoie les données de la réservation créée, incluant `id_reservations`.

   ```
   { "id_reservations": 42, "montant_total": 735.00, ... }
   ```

   - L'utilisateur est redirigé vers l'écran de **confirmation** (`BookingSuccessScreen`).
7. **Si une erreur survient** (dates indisponibles, bien inexistant, etc.) :
   - Le message d'erreur traduit est affiché dans le formulaire.

Écran de confirmation (`BookingSuccessScreen`)

- Affiche une icône de validation, le nom du bien, le montant total, et le numéro de réservation.
- Un message informe l'utilisateur qu'un email de confirmation a été envoyé.
- Le bouton **« Retour à l'accueil »** redirige vers `MainNavScreen`.
- **Aucun appel API** sur cet écran.

```
┌─────────────────┐                                    ┌──────────────┐
│ BookingScreen   │                                    │   Serveur    │
│   Étape 1       │  GET /biens/5/disponibilite        │   API REST   │
│                 │  ?date_debut=...&date_fin=...      │              │
│                 │ ──────────────────────────────────> │  Vérifie     │
│                 │  { disponible: true }              │  blocages +  │
│                 │ <────────────────────────────────── │  réservations│
│   Étape 2       │  (récapitulatif local)             │              │
│   Étape 3       │  POST /reservations                │              │
│                 │  { date_debut, date_fin,           │  Calcule le  │
│                 │    id_bien, id_tarif }             │  montant et  │
│                 │ ──────────────────────────────────> │  crée en BDD │
│                 │  { id_reservations: 42, ... }      │              │
│                 │ <────────────────────────────────── │              │
│                 │──> BookingSuccessScreen            │              │
└─────────────────┘                                    └──────────────┘
```

8 — Favoris

Écrans : `HomeScreen`, `MapScreen`, `FavoritesScreen`
Fichier : `lib/screens/favorites_screen.dart`
Service : `FavoriteService` (`lib/services/favorite_service.dart`)

8.1 — Chargement de la liste des favoris

1. L'utilisateur accède à la page **« Mes favoris »** via la barre de navigation.
2. L'application effectue un appel `GET /nestvia/favoris`.

```
GET /nestvia/favoris
Authorization: Bearer eyJ...
  → 200 : [ { id_favoris: 10, id_bien: 3, nom_bien: "...", ... }, ... ]
```

3. Les biens reçus sont transformés en objets `Property` puis enrichis (photos + tarifs) via `PropertyService.enrichProperties()`.
4. La liste des favoris est affichée sous forme de cartes.

8.2 — Ajout d'un favori (toggle ON)

1. L'utilisateur clique sur l'icône cœur vide d'un bien (depuis HomeScreen, MapScreen, ou FavoritesScreen).
2. **Mise à jour optimiste** : l'interface bascule immédiatement le cœur en rouge (sans attendre la réponse API).
3. L'application effectue un appel `POST /nestvia/favoris` en arrière-plan :

```
POST /nestvia/favoris
Authorization: Bearer eyJ...
Content-Type: application/json

{
  "id_bien": 5
}
```

4. **Si l'appel réussit** (code `200` ou `201`) :
   - L'API renvoie l'ID du favori créé.

   ```
   { "id_favoris": 15 }
   ```

   - L'ID est stocké dans le cache local.
5. **Si l'appel échoue** :
   - L'état local est **réverté** (le cœur redevient vide).

8.3 — Suppression d'un favori (toggle OFF)

1. L'utilisateur clique sur l'icône cœur plein d'un bien.
2. **Mise à jour optimiste** : le cœur devient immédiatement vide.
3. L'application effectue un appel `DELETE /nestvia/favoris/:id_bien` en arrière-plan :

```
DELETE /nestvia/favoris/5
Authorization: Bearer eyJ...
  → 200 : (succès)
```

4. **Si l'appel échoue** :
   - L'état local est **réverté** (le cœur redevient plein).

---

9 — Mes réservations

Écran : `MyReservationsScreen`
Fichier : `lib/screens/my_reservations_screen.dart`
Service : `ProfileService` (`lib/services/profile_service.dart`)

Flux détaillé

1. L'utilisateur accède à **« Mes réservations »** depuis le profil.
2. L'application effectue un appel `GET /nestvia/reservations`.

```
GET /nestvia/reservations
Authorization: Bearer eyJ...
  → 200 : [
    {
      "id_reservations": 42,
      "date_debut": "2026-07-05",
      "date_fin": "2026-07-19",
      "id_bien": 5,
      "nom_bien": "Villa Soleil",
      "montant_total": 735.00,
      ...
    },
    ...
  ]
```

3. Les réservations sont classées **localement** par statut (calculé à partir des dates) :
   - **À venir** : `date_debut` > aujourd'hui
   - **En cours** : `date_debut` ≤ aujourd'hui ≤ `date_fin`
   - **Terminée** : `date_fin` < aujourd'hui
4. L'utilisateur peut filtrer par statut via des chips de filtre (filtrage local, pas d'appel API).
5. Le pull-to-refresh recharge les réservations depuis l'API.

---

10 — Notifications

Écran : `NotificationsScreen`
Fichier : `lib/screens/notifications_screen.dart`
Service : `NotificationService` (`lib/services/notification_service.dart`)

10.1 — Chargement des notifications

1. L'utilisateur accède à la page **Notifications**.
2. L'application effectue un appel `GET /nestvia/notifications`.

```
GET /nestvia/notifications
Authorization: Bearer eyJ...
  → 200 : [
    {
      "id_notification": 1,
      "type": "reservation",
      "message": "Votre réservation a été confirmée.",
      "is_read": 0,
      "created_at": "2026-03-10T14:30:00Z",
      ...
    },
    {
      "id_notification": 2,
      "type": "review_request",
      "message": "Donnez votre avis sur votre séjour !",
      "is_read": 0,
      "id_reservation": 42,
      ...
    },
    ...
  ]
```

3. Les notifications sont affichées avec un badge indiquant le nombre de non-lues.

10.2 — Marquer une notification comme lue

1. L'utilisateur interagit avec une notification (clic/swipe).
2. L'application effectue un appel `PATCH /nestvia/notifications/:id/read`.

```
PATCH /nestvia/notifications/1/read
Authorization: Bearer eyJ...
  → 200 : (succès)
```

3. L'état local de la notification passe à `is_read: 1`.

10.3 — Marquer toutes les notifications comme lues

1. L'utilisateur clique sur **« Tout marquer comme lu »**.
2. L'application effectue un appel `PATCH` **pour chaque notification non lue** (séquentiellement) :

```
PATCH /nestvia/notifications/1/read → 200
PATCH /nestvia/notifications/2/read → 200
...
```

3. Toutes les notifications passent à `is_read: 1` dans l'état local.

---

11 — Dépôt d'avis

Widget : `ReviewDialog`
Fichier : `lib/widgets/review_dialog.dart`
Service : `NotificationService`, `ProfileService`

Flux détaillé

1. L'utilisateur reçoit une notification de type `review_request` (après un séjour terminé).
2. L'utilisateur clique sur cette notification → ouverture de la boîte de dialogue `ReviewDialog`.
3. L'application charge le détail de la réservation pour obtenir l'`id_bien` associé :

```
GET /nestvia/reservations/42
Authorization: Bearer eyJ...
  → 200 : { "id_reservations": 42, "id_bien": 5, ... }
```

4. L'utilisateur sélectionne une note de 1 à 5 étoiles et rédige un commentaire (optionnel).
5. L'utilisateur clique sur **« Envoyer »**.
6. L'application effectue un appel `POST /nestvia/biens/:id_bien/avis` :

```
POST /nestvia/biens/5/avis
Authorization: Bearer eyJ...
Content-Type: application/json

{
  "id_reservation": 42,
  "rating": 4,
  "comment": "Très bel endroit, je recommande !"
}
```

7. **Si l'avis est créé avec succès** (code `200` ou `201`) :
   - La boîte de dialogue se ferme.
   - La notification est marquée comme lue.
   - Un message de confirmation *« Merci pour votre avis ! »* s'affiche.
8. **Si une erreur survient** :
   - Le message d'erreur traduit est affiché dans la boîte de dialogue.

---

12 — Profil utilisateur

Écran : `ProfileScreen`
Fichier : `lib/screens/profile_screen.dart`
Service : `ProfileService` (`lib/services/profile_service.dart`)

Flux détaillé

12.1 — Chargement du profil

1. L'utilisateur accède à l'onglet **Profil**.
2. L'application effectue **deux appels séquentiels** :

```
GET /nestvia/compte
Authorization: Bearer eyJ...
  → 200 : {
    "nom_locataire": "Dupont",
    "prenom_locataire": "Jean",
    "email_locataire": "jean@exemple.com",
    "dna_locataire": "2024-01-15",
    ...
  }
```

```
GET /nestvia/reservations
Authorization: Bearer eyJ...
  → 200 : [ { id_reservations: 42, date_fin: "2026-01-10", ... }, ... ]
```

3. L'écran affiche :
   - **En-tête** : initiales, nom complet, ancienneté (calculée à partir de `dna_locataire`).
   - **Statistiques** : nombre total de réservations + nombre de séjours terminés (calculés localement à partir des dates).
   - **Menus** : informations personnelles, paiements, sécurité, notifications, réservations, favoris, aide, paramètres.

12.2 — Mise à jour du profil (structure prévue)

Le service expose la méthode `updateProfile()` qui effectue :

```
PUT /nestvia/compte
Authorization: Bearer eyJ...
Content-Type: application/json

{
  "nom_locataire": "Dupont",
  "prenom_locataire": "Jean",
  "email_locataire": "jean@nouveau.com",
  "tel_locataire": "0612345678"
}
```

Champs modifiables côté API : `nom_locataire`, `prenom_locataire`, `dna_locataire`, `email_locataire`, `rue_locataire`, `tel_locataire`, `comp_locataire`, `id_commune`, `raison_sociale`, `siret`, `password`.

---

13 — Déconnexion

Écran : `ProfileScreen`
Fichier : `lib/screens/profile_screen.dart`
Services : `AuthService`, `FavoriteService`

Flux détaillé

1. L'utilisateur clique sur le bouton **« Déconnexion »** sur la page profil.
2. Une boîte de dialogue de confirmation s'affiche : *« Êtes-vous sûr de vouloir vous déconnecter ? »*
3. L'utilisateur confirme.
4. L'application effectue les opérations suivantes **localement** (aucun appel API) :
   - Suppression du token JWT du stockage sécurisé (`FlutterSecureStorage.delete(key: 'auth_token')`).
   - Suppression du header `Authorization` du client Dio (`ApiService().clearAuthToken()`).
   - Vidage du cache local des favoris (`FavoriteService().clear()`).
5. L'utilisateur est redirigé vers la **page d'accueil non-connectée** (`LandingScreen`).
6. Toute la pile de navigation est réinitialisée (`pushAndRemoveUntil`).

> **Aucun appel API n'est effectué** pour la déconnexion. Le token JWT expire naturellement après 24h côté serveur. La déconnexion est strictement côté client.

---

14 — Synthèse des endpoints

Endpoints API utilisés par l'application

Méthode	Endpoint	Auth	Écran(s) déclencheur(s)	Description
`POST`	`/nestvia/auth/login`	Non	AuthScreen	Connexion (email + password) → Token JWT
`GET`	`/nestvia/biens`	Oui	HomeScreen, MapScreen, SearchScreen	Liste des biens (avec filtres optionnels)
`GET`	`/nestvia/biens/:id`	Oui	PropertyPopup (MapScreen)	Détail d'un bien + prestations
`GET`	`/nestvia/biens/:id/photos`	Oui	HomeScreen, MapScreen, BookingScreen, PropertyPopup	Photos d'un bien

Méthode	Endpoint	Auth	Écran(s) déclencheur(s)	Description
GET	/nestvia/biens/:id/tarifs	Oui	HomeScreen, MapScreen, BookingScreen	Tarifs d'un bien
GET	/nestvia/biens/:id/disponibilite	Oui	SearchScreen, BookingScreen	Vérification de disponibilité sur une période
POST	/nestvia/biens/:id/avis	Oui	ReviewDialog (NotificationsScreen)	Création d'un avis (rating + commentaire)
GET	/nestvia/communes	Oui	SearchScreen	Recherche de communes (autocomplétion)
GET	/nestvia/types-bien	Oui	HomeScreen, SearchScreen	Liste des types de biens
GET	/nestvia/favoris	Oui	HomeScreen, FavoritesScreen	Liste des favoris de l'utilisateur
POST	/nestvia/favoris	Oui	HomeScreen, MapScreen, FavoritesScreen	Ajout d'un bien aux favoris
DELETE	/nestvia/favoris/:id_bien	Oui	HomeScreen, MapScreen, FavoritesScreen	Suppression d'un favori
GET	/nestvia/reservations	Oui	MyReservationsScreen, ProfileScreen	Liste des réservations de l'utilisateur
GET	/nestvia/reservations/:id	Oui	ReviewDialog	Détail d'une réservation
POST	/nestvia/reservations	Oui	BookingScreen (Étape 3)	Création d'une réservation
GET	/nestvia/notifications	Oui	HomeScreen, NotificationsScreen	Liste des notifications
PATCH	/nestvia/notifications/:id/read	Oui	NotificationsScreen	Marquer une notification comme lue
GET	/nestvia/compte	Oui	HomeScreen, ProfileScreen	Informations du compte connecté

Méthode	Endpoint	Auth	Écran(s) déclencheur(s)	Description
PUT	/nestvia/compte	Oui	ProfileScreen	Mise à jour des informations du compte

URLs externes (hors API, ouvertes dans le navigateur)

URL	Écran déclencheur	Usage
https://nestvia.leofranz.fr/connexion_inscription.php	LandingScreen, AuthScreen	Inscription / Mot de passe oublié

Schéma global des flux

```
                                    ┌──────────────────────────┐
                                    │    API REST (Serveur)    │
                                    │  api.leofranz.fr/nestvia │
                                    └──────────────┬───────────┘
                                                   │
┌──────────────────────────────────────────────────┼──────────────────────────────────────────┐
          │                                        │
│
    ┌─────▼──────┐  POST /auth/login      ┌─────▼──────┐  GET /biens, etc.
┌─────▼──────┐
    │  Landing   │ ──────────────────────> │  Auth      │ ──────────────────────>
│  Home    │
    │  Screen    │                         │  Screen    │  (avec Bearer JWT)
│  Screen  │
    └────────────┘                         └────────────┘
└────┬───────┘

│

┌──────────────────┬──────────────────┬──────────────────┬──────────────────┬──────────────────┤
          │                  │                  │                  │
│
    ┌─────▼──────┐  ┌──────────▼─┐  ┌──────────▼─┐  ┌──────────▼─┐
┌──────────▼─┐
    │  Search    │  │    Map     │  │  Favorites  │  │  Notifs    │  │
Profile    │
    │  Screen    │  │   Screen   │  │   Screen   │  │  Screen    │  │
Screen     │
    └────────────┘  └────────────┘  └────────────┘  └────────────┘
└────────────┘
          │                │                │                │                │
          │          ┌─────▼──────┐         │          ┌─────▼──────┐
┌─────▼──────┐
```

```
          |         |  Booking  |         |         |  Review     |  |  Mes
résas  |
          |         |  Screen   |         |         |  Dialog     |  |
Screen    |
          └─────────┤ (3 étapes)|         |         └─────────────┘
└─────────────┘
                    └─────┬─────┘         |
                          |               |
                    ┌─────▼─────┐         |
                    |  Success  |         |
                    |  Screen   |         |
                    └───────────┘         |
```

Gestion des erreurs réseau (toutes les interactions)

Chaque appel API passe par le client Dio centralisé (`ApiService`). En cas d'erreur, le traitement est uniforme :

Type d'erreur	Code HTTP	Message affiché à l'utilisateur
Timeout (connexion/envoi/réception)	—	« La connexion a expiré. Vérifiez votre réseau. »
Erreur de connexion	—	« Impossible de contacter le serveur. Vérifiez votre connexion internet. »
Identifiants incorrects / session expirée	401	« Identifiants incorrects ou session expirée. »
Ressource introuvable	404	« Ressource introuvable. » (ou message API)
Conflit (doublon)	409	« Conflit : la ressource existe déjà. » (ou message API)
Trop de requêtes	429	« Trop de requêtes. Veuillez réessayer plus tard. »
Erreur serveur	5xx	« Le serveur est temporairement indisponible. » (ou message API)

Sécurité des échanges

- **HTTPS** : toutes les communications sont chiffrées (TLS).
- **JWT** : le token est signé côté serveur avec `JWT_SECRET` (algorithme HS256) et expire après 24h.
- **Bearer** : le token est transmis dans le header `Authorization: Bearer <token>` à chaque requête authentifiée.
- **Stockage sécurisé** : le token est stocké via `FlutterSecureStorage` (Android Keystore / iOS Keychain).
- **Rate limiting** côté serveur : 20 requêtes / 15 min sur le login, 50 / 15 min sur les tentatives.
- **Protection injection SQL** côté serveur : requêtes paramétrées.
- **Helmet** côté serveur : headers de sécurité HTTP.